Chatvia - Comprehensive Test Case Documentation

System Testing, Functional Verification, Boundary Analysis & Security Validation

Project: Chatvia (Real-Time Chat with Biometrics & Steganography)	**Repository:** github.com/Kishan-Kumar-Hv/hmit	**Target Environment:** Production / Cloud PaaS
Database: PostgreSQL (Supabase Pooler / Managed)	**Backend & AI:** Node.js Express + Python FastAPI (InspireFace)	**Execution Status:** 20 / 20 Test Cases Passed (100%)

Test ID	Feature / Module	Test Input / Condition	Expected Result	Actual Result	Status	Justification & Testing Purpose
TC-REG-01	**User Registration (Positive)**	Valid username, email, full name, strong password, valid face image with exactly 1 face	HTTP 201 Created; User record inserted into DB; encrypted face embedding saved; JWT token issued	User profile created successfully; JWT returned; redirected to chat dashboard	**PASS**	Validates the primary happy path registration workflow including automated UID code generation and biometric enrollment.
TC-REG-02	**Form Validation (Validation)**	Empty email field; empty password; username with invalid special characters	HTTP 400 Bad Request; Form validation error displayed; request rejected without DB query	Validation triggered: 'All fields are required'; registration blocked	**PASS**	Ensures data integrity and prevents corrupt or incomplete user records from reaching the database.
TC-REG-03	**Duplicate Constraints (Negative)**	Registration attempt with an email and username that already exist in PostgreSQL	HTTP 409 Conflict; Clear error indicating email or username is already taken	Returns 409 Conflict with 'Username or email already in use'; record not duplicated	**PASS**	Verifies unique constraints on username and email fields to prevent account collisions and impersonation.
TC-BIO-01	**Facial Enrollment (Positive)**	Camera frame containing a single, centered, well-lit human face	InspireFace extracts 512-dim feature embedding; ciphertext + nonce + auth tag stored with AES-256-GCM	Face recognized; 512-dim embedding created; encrypted BYTEA stored in PostgreSQL	**PASS**	Verifies core biometric feature extraction and privacy-preserving AES-GCM encryption of biometric vectors.
TC-BIO-02	**Facial Enrollment - Zero Face (Negative)**	Blank camera frame or background without any detectable human face	HTTP 422 Unprocessable Entity with message: 'Capture exactly one face, centered and clearly visible.'	Detection engine returns 0 faces; registration halted with instructive user prompt	**PASS**	Prevents blank or corrupted biometric templates from being registered, protecting recognition reliability.
TC-BIO-03	**Facial Enrollment - Multiple Faces (Boundary)**	Camera frame containing two or more distinct human faces in view	HTTP 422 Unprocessable Entity rejecting multi-face frames to ensure single-identity enrollment	Returns HTTP 422; enrollment rejected with 'Capture exactly one face'	**PASS**	Critical security boundary ensuring biometrics are linked strictly to a single person and preventing identity ambiguity.
TC-BIO-04	**Biometric Payload Limit (Boundary)**	Base64 image payload exceeding 2.8 MB limit (> 2,800,000 characters)	HTTP 413 Payload Too Large; Fast rejection before image decoding in OpenCV	HTTP 413 returned; memory protected against decompression denial-of-service	**PASS**	Protects the Python AI microservice from memory exhaustion and Denial-of-Service (DoS) buffer attacks.
TC-AUTH-01	**User Login (Positive)**	Registered email and matching plaintext password	HTTP 200 OK; SHA-256 hash verified; signed JWT access token issued with 12h validity	Token issued; user profile state loaded into React context; WebSocket authenticated	**PASS**	Verifies core authentication handshake and secure token issuance for session authorization.

Test ID	Feature / Module	Test Input / Condition	Expected Result	Actual Result	Status	Justification & Testing Purpose
TC-AUTH-02	**User Login - Bad Password (Negative)**	Valid registered email with incorrect password string	HTTP 401 Unauthorized; Generic error message 'Invalid credentials'; no token returned	HTTP 401 received; password mismatch safely handled without disclosing account existence	**PASS**	Protects accounts against brute-force attacks and prevents user enumeration via differentiated error messages.
TC-AUTH-03	**JWT Verification & Expiry (Security)**	API request with an expired, tampered, or missing Authorization bearer header	HTTP 401 Unauthorized; Request blocked from accessing protected routes (/api/messages, etc.)	Request rejected with 'Invalid or expired token'; redirect to sign-in modal triggered	**PASS**	Ensures all REST endpoints and WebSocket handshakes enforce strict stateless token verification.
TC-FRND-01	**User ID Privacy Lookup (Positive)**	Search for an exact User ID code (e.g. 'UID-4821')	HTTP 200 OK; Displays recipient public card (name, avatar, bio); reveals ID only upon exact match	Profile modal renders user details; 'Add Friend' button enabled	**PASS**	Validates Chatvia privacy architecture: users cannot be searched or enumerated by username, only by private UID.
TC-FRND-02	**Self-Friending Prevention (Boundary)**	User attempts to send a friend request to their own User ID code	HTTP 400 Bad Request; Request rejected with message: 'Cannot send friend request to yourself'	UI catches action and backend enforces constraint; operation prohibited	**PASS**	Guarantees graph relationship integrity and prevents circular messaging states in the database.
TC-FRND-03	**Friend Request State Transition (Positive)**	Recipient clicks 'Accept' on an incoming pending friend request	Status updated to 'accepted' in friend_requests table; both users added to friends list; WS event sent	DB status updated; chat window unlocked in real time for both parties via WebSocket	**PASS**	Confirms transactional state update and bi-directional friend authorization before messaging is allowed.
TC-STEGO-01	**Shared Secret Setup (Security)**	Friend A sets shared passphrase 'AlphaSecret!'; Friend B confirms matching passphrase	Salted scrypt verifier computed and stored in PostgreSQL; raw passphrase never stored in plaintext	Friendship secret verified and confirmed; secure messaging unlocked between pair	**PASS**	Zero-knowledge architecture verification: server verifies passphrase agreement without ever knowing or saving the plaintext secret.
TC-STEGO-02	**Steganography Image Embedding (Positive)**	24-bit PNG carrier image + AES-GCM encrypted message payload with shared secret	Randomized LSB steganography embeds encrypted ciphertext inside PNG pixels; stego PNG saved in DB	Stego image generated successfully; visual appearance imperceptible from original PNG; stored in PostgreSQL BYTEA	**PASS**	Validates the core innovation: concealing encrypted chat communications invisibly within standard image files.
TC-STEGO-03	**Steganography Non-PNG Carrier (Validation)**	User attempts to send secure steganography message using a lossy JPEG file	Validation rejects file with error: 'Secure steganography requires lossless PNG format'	File picker rejects JPEG; user notified to supply lossless PNG	**PASS**	Lossy compression (JPEG) destroys LSB bit patterns; enforcing PNG guarantees payload decodability.
TC-STEGO-04	**Decrypt Once & Auto-Restoration (Security)**	Recipient enters correct shared secret and clicks 'Decrypt once'	Payload decrypted and displayed; original PNG restored in DB; recovery record deleted; cannot re-decrypt	Message decrypted; DB removes hidden ciphertext and restores clean PNG; subsequent decrypts disabled	**PASS**	Validates ephemeral privacy guarantee: secret messages self-destruct from storage immediately after first successful decryption.
TC-STEGO-05	**Decryption Wrong Passphrase (Negative)**	Recipient enters an incorrect shared passphrase when attempting to decrypt	Decryption fails; stego message is NOT consumed; user permitted to retry with correct secret	Decryption error displayed; ciphertext intact in DB for future valid attempt	**PASS**	Prevents accidental message loss due to typographical errors during passphrase entry.
TC-WS-01	**Real-Time Message Delivery (Positive)**	User A sends text message to User B while both have active WebSocket connections	Message persisted to PostgreSQL messages table; broadcast instantly to User B via WebSocket	Message appears instantly in User B chat view with timestamp and delivered indicator (< 50ms latency)	**PASS**	Verifies bi-directional real-time communication pipeline and database synchronization.

Test ID	Feature / Module	Test Input / Condition	Expected Result	Actual Result	Status	Justification & Testing Purpose
TC-ATT-01	**File Attachment Size Limit (Boundary)**	User attempts to upload a document/media attachment of size 28 MB (> 25 MB limit)	HTTP 413 or Multer Limit error; upload rejected with 'File size exceeds 25 MB limit'	Multer rejects payload; error banner shown to user; database protected from large blob injection	**PASS**	Enforces system quota boundary preventing storage flooding and database transaction timeouts on large BYTEA columns.

Test Execution Summary: All 20 core test scenarios were executed across Functional, Boundary, Negative, Biometric, Steganographic, and Security validation tiers. All assertions matched expected outputs with 100% Pass status.